UNIVERSIDAD DE MANILA • COLLEGE OF COMPUTING STUDIES bikko.studio
Human Computer Interaction (ITE 323) • Prof. Mary Grace Nulud Academic Year 2026

Technical Documentation and Member Assignment: Carl Adrian A. Tan

- **Member Name**: Carl Adrian A. Tan (Adrian)
- **Assigned Role**: Role 2: System Framework (Backend Architecture, WebAuthn, Security and PWA Developer)
- **Project Title**: bikko.studio (System Architecture: studioNeo / sankobite-v3)
- **Institution**: Universidad de Manila, College of Computing Studies
- **Course**: Human Computer Interaction (ITE 323)

1. Role Overview and Scope of Work

Adrian serves as the Backend Architecture, WebAuthn, Security, and Progressive Web Application (PWA) Developer. His scope of work encompasses the core Next.js 16 server framework, hardware-bound passwordless authentication (FIDO2 / WebAuthn Level 3), zero-knowledge fallback login pipelines, single sign-on (SSO) integrations, security middleware, and offline service worker caching engines.

His primary objective is to deliver a rock-solid, zero-password cloud backend architecture that authenticates users in under 250 milliseconds, defends against credential theft and timing attacks, and ensures offline system resilience.

2. Core Technical Responsibilities

2.1. Core Framework and Speed Optimization

1. **Next.js 16 App Router Architecture (NFR6 - Clean Code Architecture)**:
2. Architect the entire application using strict TypeScript and React 19 Server Components within the Next.js 16 App Router framework.
3. Enforce a clean architectural separation between server-rendered layouts and client interactive widgets:
    - Server Components handle database queries, authentication verification, and initial HTML generation on the server.
    - Client Components (annotated with `"use client"`) handle local state, touch gestures, and canvas animations.
4. Core files: `src/app/layout.tsx`, `src/app/page.tsx`, `src/app/api/auth/*`.
5. **Performance and Load Budgets (NFR1 - Speed and Performance Efficiency)**:
6. Optimize page delivery pipelines so First Contentful Paint (FCP) and Largest Contentful Paint (LCP) complete within 1.2 seconds on broadband internet and within 2.2 seconds on standard 4G mobile networks.
7. Minimize client-side bundle sizes by avoiding heavy third-party authentication client libraries, relying on native browser WebAuthn APIs and lightweight server actions.
8. Maintain sub-100 millisecond response times for user clicks and touch events through asynchronous background processing.

2.2. Passwordless FIDO2 / WebAuthn and Backup Authentication

1. **Biometric Passkey Flow (FR1 - Passkey Registration and Biometric Verification)**:
2. Implement W3C WebAuthn Level 3 cryptographic standards using `@simplewebauthn/server` and `@simplewebauthn/browser`.
3. **Challenge Generation**: The server creates high-entropy cryptographic challenges using Node.js `crypto.randomBytes(32)` with a 60-second expiration window to prevent replay attacks.
4. **Hardware Signature Verification**: Verify public key credentials signed by device hardware chips (Apple Secure Enclave, Android StrongBox, Windows Hello TPM, or external YubiKey hardware tokens).
5. Supports both user-verifying platform authenticators (Face ID, Touch ID, Windows Hello) and roaming security keys.
6. Core files: `src/lib/webauthn.ts`, `src/app/api/auth/passkey/login-options/route.ts`, `src/app/api/auth/passkey/login-verify/route.ts`, `src/app/api/auth/passkey/register-options/route.ts`, `src/app/api/auth/passkey/register-verify/route.ts`.
7. **Sub-250ms Sign-in Response**:
8. Streamline cryptographic challenge generation, signature validation, and database key retrieval to transition the user to their authenticated feed in under 250 milliseconds (benchmarked at approximately 180 milliseconds under normal network conditions).
9. **Password-Free Backup Sign-in (FR2 - Password-Free Backup Sign-in)**:
10. **6-Digit Email Access Code**: For users signing in from a new or unlinked device, generate a cryptographically random 6-digit numeric code valid for exactly 10 minutes. The plain code is dispatched via email while its secure SHA-256 hash is recorded in the database.
11. **RFC 6238 TOTP Authenticator Support**: Implement Time-Based One-Time Password verification compatible with Google Authenticator, Microsoft Authenticator, and 1Password, using 30-second time steps and drift tolerance.
12. **Hashed Single-Use Recovery Keys**: Provide users with single-use emergency backup recovery keys stored as one-way cryptographic hashes.
13. Core files: `src/lib/auth.ts`, `src/lib/totp.ts`, `src/lib/email.ts`, `src/app/api/auth/login/route.ts`, `src/app/api/auth/verify-otp/route.ts`.
14. **Social Single Sign-On (FR3 - Social Media Single Sign-On)**:
15. Configure federated OAuth 2.0 / OpenID Connect (OIDC) authentication supporting Google Identity Services and Microsoft Entra ID.
16. Enforce Proof Key for Code Exchange (PKCE) with code verifiers and SHA-256 code challenges, plus cryptographic state parameters to prevent Cross-Site Request Forgery (CSRF).
17. Automatically initialize a member profile and handle when a new user signs in for the first time.
18. Core files: `src/lib/oauth.ts`, `src/app/api/auth/oauth/[provider]/route.ts`, `src/app/api/auth/oauth/[provider]/callback/route.ts`.

2.3. Security Middleware and PWA Offline Engine

1. **Hardened Session Cookies (NFR2 - Security and Protection Standards)**:
2. Issue session tokens using encrypted HTTP cookies protected with industry-standard security flags:
    - `HttpOnly`: Prevents client-side JavaScript from accessing session tokens, eliminating Cross-Site Scripting (XSS) token theft.

- `SameSite=Lax` / `SameSite=Strict`: Protects against Cross-Site Request Forgery (CSRF) by preventing browsers from attaching authentication cookies to unauthorized third-party requests.
- `Secure`: Restricts cookie transmission strictly to encrypted TLS connections.

3. Core file: `src/lib/auth.ts`.
4. **Timing-Attack Defense (NFR2 - Security and Protection Standards)**:
5. Use constant-time comparison algorithms (`crypto.timingSafeEqual`) whenever verifying OTP codes, digital signatures, session tokens, or API secrets.
6. Prevents attackers from using slight variations in response times to guess authentication secrets byte by byte.
7. Implementation example: ```typescript import crypto from "crypto";

   export function safeCompare(a: string, b: string): boolean { const bufA = Buffer.from(a, "utf-8"); const bufB = Buffer.from(b, "utf-8"); if (bufA.length !== bufB.length) { return false; } return crypto.timingSafeEqual(bufA, bufB); } ```
8. **Progressive Web App Service Worker Caching (FR8 - PWA Offline Support)**:
9. Register a background Service Worker caching core static assets for instant loading and home-screen installation:
   - Application layout shell (`/`)
   - Web fonts and icon glyphs
   - Vector SVG files and artwork data
   - Compiled CSS style sheets and JavaScript bundles
10. Core files: `public/sw.js`, `src/app/manifest.ts`, `src/app/components/ServiceWorkerRegister.tsx`.
11. **Offline Fallback Handler (NFR5 - Reliability and Offline Grace)**:
12. Detect lost internet connectivity using browser event listeners (`navigator.onLine`, `window.addEventListener('offline')`).
13. Intercept failed network requests in the Service Worker and serve cached pages from local storage.
14. Trigger a clean, friendly offline notification banner across the UI without application crashes or white screen freezes.

---

3. Documentation Sections Owned

Adrian is directly responsible for drafting, updating, and maintaining the following formal sections in the project report:

1. **Section 2.3.1: Functional Requirements (FR1, FR2, FR3, FR8)**:
2. FR1: Passkey Registration and Biometric Verification (WebAuthn Level 3, challenge generation, signature checks, sub-250ms latency).
3. FR2: Password-Free Backup Sign-in (10-minute 6-digit email OTPs, RFC 6238 TOTP authenticator verification, single-use recovery keys).
4. FR3: Social Media Single Sign-On (Google and Microsoft OAuth 2.0 with PKCE and auto profile creation).
5. FR8: Progressive Web App Offline Support (Service worker caching, home-screen manifest, offline app shell).
6. **Section 2.3.2: Non-Functional Requirements (NFR1 to NFR6)**:
7. NFR1: Speed and Performance Efficiency (1.2s fast connection, 2.2s 4G mobile, sub-100ms click latency, 0% idle CPU).

8. NFR2: Security and Protection Standards (HttpOnly SameSite Secure cookies, constant-time byte comparisons, TLS encryption).
9. NFR3: Universal Accessibility (System support for WCAG AA standards and high contrast).
10. NFR4: Cross-Device Responsiveness (Mobile, tablet, desktop, and 4K display support).
11. NFR5: Reliability and Offline Grace (Offline network fallback banner, graceful error handling).
12. NFR6: Clean Code Architecture (Strict TypeScript, React 19 Server Components, Next.js 16 App Router).
13. **Section 2.3.4: Security Protection Overview (Multi-Layer Security Defense)**:
14. Document the multi-layer security architecture:
    - Layer 1: Hardware Sign-in via WebAuthn Biometrics and Roaming Security Keys.
    - Layer 2: Data Transport via TLS Encrypted Connections.
    - Layer 3: Session Safety via HttpOnly, SameSite, and Secure Cookies.
    - Layer 4: Timing-Attack Defense using equal-time byte checks.
    - Layer 5: Cloud File Rules using 15-minute temporary read-only download links.
15. **Section 3.3.5: Progressive Web Application (PWA) Offline Support and Memory System**:
16. Document the Service Worker cache architecture, cache-first routing rules, offline asset fallback flows, and device memory management.

---

4. Cross-Role Integration and Development Handoffs

- **Handoff to Role 1 (Jaz - Frontend UI)**:
- Adrian supplies the authentication endpoints (`/api/auth/passkey/login-options` , `/api/auth/passkey/login-verify` , `/api/auth/login` , `/api/auth/verify-otp`) to Jaz's Sign-in Gateway modal dialog.
- Adrian creates the Service Worker registration and network offline broadcast events that toggle Jaz's offline UI banner.
- **Handoff to Role 3 (Gwyneth - Database and Queries)**:
- Adrian passes verified user credentials, public keys, authenticator registration data, and OTP hashes to Gwyneth's MySQL tables (`webauthn_credentials` , `auth_magic_tokens` , `totp_credentials` , `totp_backup_codes`).
- Adrian uses Gwyneth's subscriber entitlement queries to authorize user download requests before issuing signed cookie sessions.
- **Handoff to Role 4 (Paisley - Research and QA)**:
- Adrian provides working passkey and OTP authentication workflows to Paisley for end-to-end security verification and biometric sign-in testing.
- Paisley benchmarks Adrian's sign-in response times against the sub-250 millisecond performance requirement and tests offline recovery behaviors.
